

wfspan: Wait-free Dynamic Memory Management

XIANGZHEN OUYANG and YIAN ZHU, Northwestern Polytechnical University, China

Dynamic memory allocation plays a vital role in modern application programs. Modern lock-free memory allocators based on hardware atomic primitives usually provide good performance. However, threads may starve in these lock-free implementations, leading to unbounded worst-case execution time that is not allowed in real-time embedded systems. This article presents decentralized dynamic memory management, wfspan, based on non-linearizable wait-free lists. It employs a helping mechanism to ensure no starvation in the lock-free implementation. From the perspective of design tradeoff, wfspan guarantees bounded execution steps in both allocation and deallocation procedure, at the cost of increasing bounded worst-case memory footprint. The results of running benchmarks on an x86/64 and an aarch64 machine illustrate that wfspan achieves competitive performance and memory footprint compared to lock-based and lock-free practical memory allocators while showing superior to other allocators in terms of worst-case execution time.

CCS Concepts: • **Software and its engineering** → **Real-time systems software; Allocation/deallocation strategies; Embedded software**; • **Theory of computation** → *Concurrent algorithms*;

Additional Key Words and Phrases: Memory allocator, real-time systems, concurrent algorithms, lock-free, wait-free

ACM Reference format:

Xiangzhen Ouyang and Yian Zhu. 2022. wfspan: Wait-free Dynamic Memory Management. *ACM Trans. Embedd. Comput. Syst.* 21, 4, Article 43 (August 2022), 26 pages.
<https://doi.org/10.1145/3533724>

1 INTRODUCTION

Dynamic memory allocation plays a vital part in modern programming. It manages free memory and handling memory requests from user programs. In embedded real-time systems, developers usually avoid using dynamic memory allocation because of the concern that dynamic memory allocation may lead to non-deterministic execution results. In hard real-time embedded systems with partitioning strategy [38], developers usually use pre-allocated static memory management. Static memory management ensures real-time but results in a waste of memory. In most scenarios, dynamic memory allocation can better use memory resources than static pre-allocation. For example, a mobile communication server needs to handle the massive accesses from user equipment in different communication standards (e.g., 3G, 4G, or 5G), accompanied by intensive memory allocations and deallocations. If the static pre-allocation strategy is adopted, then the server should reserve memory space for each communication standard, resulting in a waste of memory and a strict limit on the maximum number of user equipment accesses. However, a real-time dynamic

Authors' address: X. Ouyang and Y. Zhu, Northwestern Polytechnical University, Xi'an, 127 West Youyi Road, Beilin District, Xi'an 710072, Shaanxi, P.R. China, 710072; emails: matrixsukhoi@mail.nwpu.edu.cn, zhuya@nwpu.edu.cn.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](https://permissions.acm.org).

© 2022 Copyright held by the owner/author(s). Publication rights licensed to ACM.

1539-9087/2022/08-ART43 \$15.00

<https://doi.org/10.1145/3533724>

Table 1. Synchronization Techniques Used in Practical Allocators

Allocators	Synchronization	Atomic Primitives	Worst-case Execution Time
Glibc	blocking/ POSIX mutex	CAS	$O(N)$ bounded (under fair scheduling)
mi	lock-free/ Treiber stack, MPSC lock-free list	SWAP, CAS2 or LL/SC	unbounded
tc	blocking/ cas spin-lock	CAS	unbounded
je	blocking/ POSIX mutex	CAS	$O(N)$ bounded (under fair scheduling)
sn	lock-free/ Treiber stack, MPSC wait-free list	SWAP, CAS2 or LL/SC	unbounded
hoard	blocking/ cas spin-lock	CAS	unbounded
super	lock-free/ transactional memory	HTM	unbounded
wfspan	wait-free/ SPMC/MPSC wait-free list	SWAP, FAA, CAS2 or LL/SC	$O(N^2)$ or $O(N)$ bounded (different helping protocol)

memory allocation used in this scenario provides elasticity. It allows more user equipment access, which is helpful for an embedded mobile communication server with limited memory.

Existing real-time dynamic memory allocation algorithms [26, 34] work well for a single-core system and provide $O(1)$ worst-case execution time for allocation and deallocation requests. However, in a multi-core system, synchronization among threads is unavoidable if they access shared resources. A shared memory allocator should introduce locks or other lock-free synchronization techniques for thread safety, which may lead to deadlock or unbounded execution time. As shown in Table 1, modern practical memory allocators use different synchronization techniques that rely on various hardware atomic primitives.

Mutex-based memory allocators such as Glibc malloc and jemalloc [8] can provide $O(N)$ bounded worst-case execution steps under fair scheduling, where N is the number of threads. However, waiting for a mutex inevitably involves context switching, scheduling, or thread migration, which are relatively complicated procedures in modern operating systems. It can increase the worst-case execution time of memory requests. To avoid context-switching, tcmalloc [10] and hoard [4] use user-level spin-lock, but these **Compare And Set– (CAS)** based spin-locks are unfair and can lead to thread starvation, which results in unbounded execution steps. What makes it even worse is the back-to-back lock acquisition [37], which can occur under heavy concurrent contention. The lock owner can release the lock and then acquires lock ownership continuously. Meanwhile, a deadlock can also occur. Even if the lock synchronization protocols and implementations in memory allocation can be formally proven correct [2, 46], since many embedded devices operate in harsh environments, a hardware error may occur when a thread holds a lock, leading to the deadlock.

The locks in dynamic memory allocators can be replaced by **Hardware Transactional Memory– (HTM)** based synchronization [17, 35] (e.g., supermalloc [20]) or lock-free data structures [7, 13, 28, 29] (e.g., mimalloc [23] and snmalloc [25]) to improve robustness. These lock-free implementations of memory allocators rely on modern hardware atomic primitives such as HTM, FAA(Fetch-And-Add), SWAP, **Load-Link/Store-Conditional (LL/SC)**, or **2-word Compare And Set (CAS2)**. Threads do not need to wait for other threads to complete but only focus on their operations. So there is no deadlock and no voluntary context switching, which can provide

a better performance, lower latency, and better robustness. However, in these lock-free implementations, threads should retry their lock-free operations until they succeed, which may cause thread starvation. There may be a much slower thread whose lock-free attempts are never successfully completed, causing non-deterministic and unbounded execution steps. In particular, in existing heterogeneous multi-core systems [3, 15] where high-performance cores run much faster than power-efficient ones, starvation is more likely to occur on the slower cores. Although thread starvation is rare in practical applications, it may cause deadline missing of a task, which is not allowed in a real-time system.

We need a stronger progress guarantee called wait-free or lock-free without starvation to ensure bounded execution steps. Many wait-free data structures have been proposed [9, 18, 19, 32, 36, 41, 44], but most have their limitations. For example, most wait-free data structures rely on additional memory reclamation mechanisms [27, 31, 42] and dynamic memory allocation [5], where the allocation itself may not be strictly wait-free. While linearizable wait-free [12] queues and stacks are extremely hard to implement, non-linearizable wait-free data structures can be implemented much simpler. Linearizability guarantees a correct concurrent history for a data structure such as a queue or stack. It ensures linearizable points that all threads can observe the result when it occurs. For example, a node that has been pushed into the queue or stack should be visible to subsequent popping or dequeuing.

Snmalloc uses a non-linearizable, high-performance **Multi-Producer Single-Consumer (MPSC)** wait-free free-list to handle remote deallocation. The remote deallocation means a thread frees memory blocks not allocated by this thread. The enqueueing of this MPSC wait-free free-list relies on atomic SWAP operation. The producer thread swaps the list's tail pointer with the newly enqueued node. Then it links the predecessor node to the new node. If a slow thread has already swapped the new node but has not completed the linking of the predecessor node yet, then the nodes inserted after this thread are invisible to the consumer threads.

From the perspective of the allocator design, using the non-linearizable wait-free free-list may lead to a waste of free memory when some slower threads prevent the reusing of all memory blocks in the free-lists. However, an upper bound on the wasted memory can be achieved via employing per-span free-lists. Here a span is the contiguous memory page with divided small memory blocks. In the worst case, a slower thread will prevent only one span from reusing. For $N - 1$ deallocating threads and one allocating thread, all deallocating threads halt before linking to the next node from different spans, preventing reusing of $N - 1$ spans. In this case, the additional memory footprint is still bounded. Using non-linearizable wait-free free-list is a tradeoff that ensures bounded worst-case execution steps at the cost of increasing the worst-case memory footprint.

The idea of using non-linearizable wait-free data structures inspired us. Based on the non-linearizable wait-free data structure, snmalloc can ensure wait-freedom in remote deallocation. However, it is challenging to ensure wait-freedom in the allocating path using existing wait-free data structures, because wait-free data structures have memory reclamation issues, leading to poor performance. This article presents wfspan, the wait-free memory management based on non-linearizable MPSC wait-free free-list and **Single-Producer Multi-Consumer (SPMC)** wait-free span-list. The non-linearizable SPMC wait-free span-list ensures bounded allocating execution steps without introducing memory reclamation. For this data structure, non-linearizability implies that one dequeuing operation may result in two nodes being dequeued. Overall, wfspan employs a decentralized architecture, where non-empty spans are published to thread own private SPMC span-lists, and the allocating thread tries to acquire spans from the span-lists of others. Both the allocation and deallocation routine are lock-free and can be completed within bounded execution steps, addressing the problem of starvation in lock-free allocators at the costs of increasing bounded worst-case memory footprint.

To evaluate wfspan, we ran micro-benchmarks and real applications workloads on a 16-core x86/64 and an 8-core heterogeneous aarch64 machine. The evaluation results demonstrate that wfspan achieves competitive performance and memory footprint, compared to practical lock-based (Glibc, tc, je, and hoard) and lock-free allocators (sn and mi). Besides, wfspan can ensure bounded execution steps in both allocation and deallocation routines. The measurement of **worst-case execution time (WCET)** in a stressed micro-benchmark shows that wfspan is significantly superior to other allocators in terms of worst-case execution time.

This article is organized as follows. In Section 2, we introduce the design considerations and overview of wfspan. In Section 3, we discuss the details of the allocator implementation. Section 4 presents the evaluation results of wfspan. Section 5 presents related work about dynamic memory allocation, and Section 6 concludes this article.

2 DESIGN

The design of memory allocators is a tradeoff. Most memory allocators avoid synchronization by thread-local allocation buffers to achieve high performance. However, using thread-local allocation buffers may cause memory waste in some cases. For bounded execution time, there is also no exception. Wfspan has the increased worst-case memory footprint in exchange for bounded execution steps in both allocation and deallocation routines. In terms of synchronization techniques, the free-lists and the span-lists used in wfspan achieve the wait-free property at the cost of losing linearizability.

In this article, we designed wfspan based on the following considerations.

Keep it Simple. Modern memory allocators typically use the design of a multi-heap layer with different allocation policies for different size classes. Instead, wfspan uses the simplified heap layer design and tries to avoid the system calls as possible. The benefits of the simplified heap layer are less worst-case execution time, simplified implementation, and less error-prone.

Linearizable wait-freedom is expensive but not necessary. Due to the limitations of current atomic primitives, linearizable wait-free is costly. In most wait-free implementations [9, 18, 19, 32, 41, 44], a helping mechanism is adopted to avoid thread starvation. Since a thread may help an expired request in the helping procedure, causing ABA problems, garbage collection or other memory reclamation techniques are required [27, 31, 42]. The throughput of a typical wait-free queue using hazard pointer as its memory reclamation technique is only about one-tenth of its lock-free counterparts. Instead, we can implement a cheaper wait-free data structure with high performance if we give up the linearizability. For example, the non-linearizable wait-free MPSC free-list used in snmalloc shows equal or better performance compared to their lock-free counterparts. Losing linearizability may cause the memory blocks managed in the data structure to be temporarily unavailable. However, we can limit unavailable memory size by partitioning strategy.

Avoid false-sharing. False-sharing occurs when multiple threads simultaneously access and modify some memory located in the same cache line, which can cause additional interconnection overhead. Like other modern memory allocators, wfspan uses proper size classes and metadata placement to avoid false-sharing.

Hide the latency. Modern hardware can hide the latency of instructions by out-of-order execution and pipeline. As a result, intuitively more expensive operations, such as helping multiple threads complete their requests or querying their private data structures, do not cost many CPU cycles on real machines.

2.1 Design Overview

For simplicity of synchronization and scalability, wfspan adopts a decentralized architecture without a globally shared heap. There are per-thread heaps that maintain two types of span-lists for

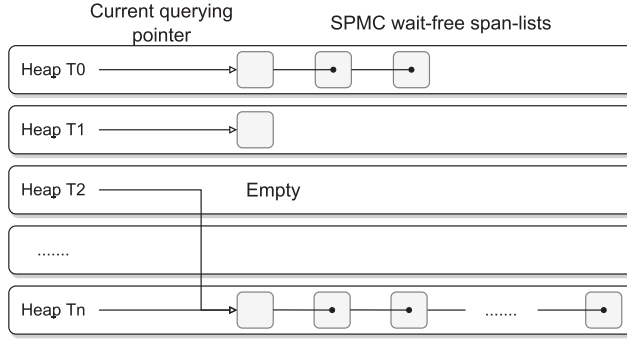


Fig. 1. Threads try acquiring a span from other threads.

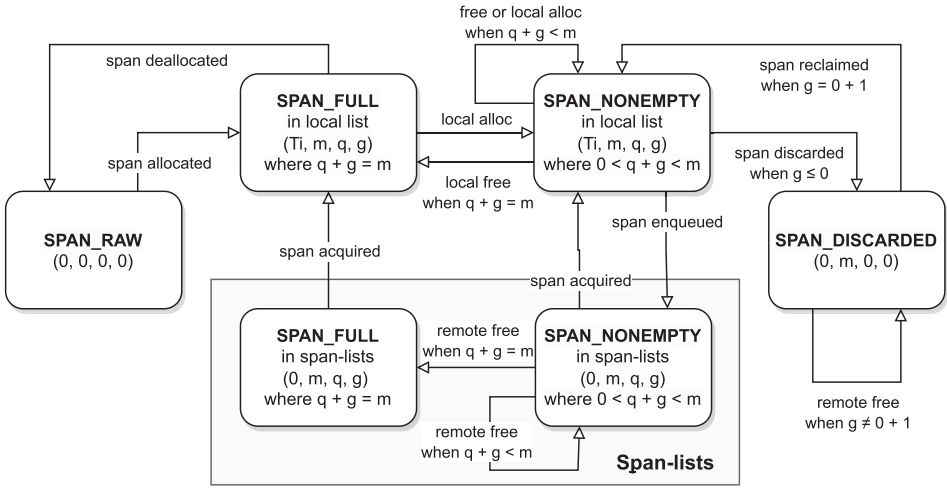


Fig. 2. The state diagram of spans.

each size class: the thread-local span-list and the SPMC wait-free span-list. The total length of thread-local span-lists has a limitation of K . The heap owner thread can push extra spans it owns into the SPMC span-lists without a lock or a lock-free attempt. The deallocations from the non-span-owner threads use the MPSC free-lists. To prevent a single heap holds unbounded free memory, threads should query the SPMC span-lists of others and tries to acquire spans in the SPMC span-lists, as shown in Figure 1. The owner thread will discard the span when its free block count is zero. After this, the first deallocating thread that adds the free block count to 1 will reclaim the span.

The main components of wfspace are shown as follows.

Spans. Span is a fixed-size contiguous memory that can be divided into small memory blocks. The span header maintains meta information such as size class, the maximum number of objects, owner thread, local free-list, remote free-list, and free block count. Like mimalloc, wfspace adopts the local free-list for local memory requests and the remote free-list for remote deallocation. The free block count represents the number of free memory blocks in the span. The free block count is increased or decreased by an atomic primitive without locking. To avoid using atomic primitives in local allocation and deallocation routine, we adopt a local free block count to track the local

effects of globally visible free block count. The local count will be added to the global count once the local free list is empty. Then, a span state can be represented as (T, m, q, g) , where T is owner thread, m is the maximum number of memory blocks in this span, q is local free block count, and g is globally visible free block count. The state diagram of spans is shown in Figure 2. The raw span allocated from the upper layer can be initialized to a full span of any size class. If at least one memory block in a full span is allocated, then the span becomes a non-empty span. The owner thread discards the span when the free block count decreases to zero. When freeing a memory block in a discarded span and the free block count increases from 0 to 1, the thread that increases the free block count will become the span owner and reclaim the span.

Heaps. Each size class of a per-thread heap has a thread-local span-list and an SPMC wait-free span-list visible to other allocators. The number of thread-owned spans is bounded by a constant parameter K . Once a per-thread heap owns more than K spans, the least recently used span will be pushed into the SPMC wait-free span-list available for other threads, as shown in Figure 2. Spans in the thread-local span-lists function as the thread-local allocation buffer.

Size-classes and pagemap. Like other modern memory allocators, wfspan uses a segregated-fit strategy. For allocation size smaller than the span size, the size classes are the same in supermalloc and smalloc. For allocation size larger than the span size, size classes are decided by the power of two. For large memory allocation, the metadata are maintained in a flat-pagemap, which is a one-level radix tree using the span index as the key.

MPSC wait-free free-list. The MPSC wait-free free-list used in smalloc can provide $O(1)$ bounded remote deallocation. In wfspan, each span contains a remote MPSC free-list for handling remote deallocations. Both the pushing and the reclaiming of the MPSC free-list rely on a single atomic SWAP instruction. This data structure is non-linearizable wait-free. If a slower thread has not completed the linking of the freed block, then all free memory blocks in an MPSC remote free-list may be unavailable. However, the partitioning strategy for the per-span remote free-list limits the worst-case memory footprint.

SPMC wait-free span-list. Non-empty spans pushed into SPMC wait-free span-lists are visible to other threads, where the producer of a span-list is only the owner thread. The pushing procedure of a span-list changes the tail pointer and links the predecessor to the new node. It requires no atomic primitive but a write memory barrier. The consumer procedure adopts a helping mechanism to ensure wait freedom. The helping protocol used here is a simplified helping protocol of KP-queue [19]. The consumer thread publishes its requests in a globally shared record data structure and then tries to help other threads complete their requests. After a consumer thread has helped several threads, it starts to finish its request. For the consumer thread, its request is either completed by itself or other threads in the helping procedure, so there is no thread starvation. The slower threads will make progress with the help of faster threads in bounded execution steps. Since we do not need to consider the scenarios under multiple producers, the data structure does not rely on additional memory reclamation mechanisms. Thanks to it, the performance of this data structure is almost the same as its lock-free counterpart. However, high performance comes at a price. For avoiding the memory reclamation issue, the consumer procedure with the helping mechanism is not linearizable. Since one dequeue request may lead to one or two spans being dequeued, it violates the definition of linearizability.

3 IMPLEMENTATIONS

In this section, we will present the details of wfspan implementation. First, we will introduce the atomic primitives used in wfspan implementations.

FAA(ptr, val). FAA atomically reads the current value old_value from memory address ptr , stores the result of $val + old_value$ to ptr , and then returns the old_val .

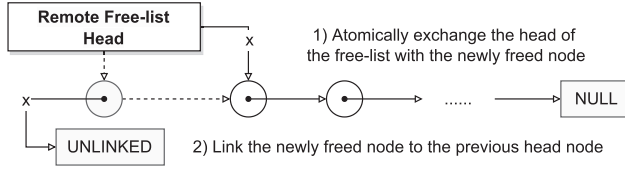


Fig. 3. The push operation of the MPSC wait-free free-list.

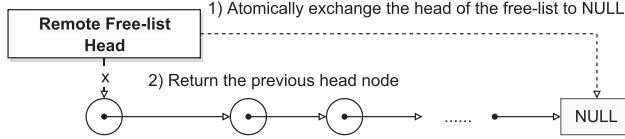


Fig. 4. The reclaim operation of the MPSC wait-free free-list.

CAS(ptr, val_ptr, new_val). CAS atomically reads current value old_val from memory address ptr and then compares it to the value read from val_ptr . If the values are equal, then new_val will be stored to ptr . And then, the old_val will be stored to val_ptr . It returns true if the operation succeeds and false if the operation fails.

SWAP(ptr, val). SWAP atomically reads its current value old_val from memory address ptr , then stores new value val into ptr and returns old_val .

LD_LINK(ptr)/ST_COND(ptr, val). LL/SC are generalized atomic operations where LD_LINK atomically marks a memory address ptr and ST_COND stores the value val to that memory address ptr . If a thread marks a memory address ptr via LD_LINK and then this memory is modified by another thread, then the next ST_COND operation of the thread will fail. Most RISC architectures support LL/SC hardware atomic primitives such as PowerPC, ARM, MIPS, and their corresponding 64-bit architectures. For the x86 architecture, we use versioned CAS2 to emulate LL/SC instructions, which extends a version field for a concurrent object. The emulated LL/SC uses a 2-word CAS to update the value and version atomically, where slower threads may fail due to the wrong versions.

ST_REL(ptr, val). ST_REL means store in released memory order. It ensures that all store instructions before it are committed.

3.1 MPSC Wait-free Free-lists

The free-lists in a span are partitioned to a local free-list and an MPSC wait-free free-list for avoiding false-sharing. Memory requests in thread-owned spans only change the local free list. The remote MPSC free-list is a non-linearizable wait-free, and the push and reclaim procedure is shown in Figures 3 and 4. The remotely deallocated memory blocks are pushed into the MPSC wait-free list. The push operation replaces the free-list head with the new node and links the node to the previous head node. The memory blocks in this list are unavailable if the nodes have not completed the node linking. When the local free-list is empty, the span owner thread tries to reclaim the remote free-list by the SWAP atomic primitive. When consuming in the local free-list, the thread will check the next of the head node. If the head node has its next node unlinked, then the consuming procedure stops, and the use of this span will delay. In the worst scenario, $N - 1$ remote free-lists of different spans cannot be available for reuse, but the additional memory footprint is bounded. We will discuss the worst-case memory footprint in Section 3.5.

3.2 SPMC Wait-free Span-lists

Each per-thread heap maintains an SPMC wait-free span-list. The span-list manages non-empty spans visible to other threads, where these spans contain at least one free memory block. The enqueueing procedure of the span-list does not require synchronization, as shown in Algorithm 1. However, in line 6, a write memory barrier is needed to ensure that all memory store operations are complete before the non-empty span is published. On the x86 ISA, the write memory barrier is not necessary, because the x86 ISA guarantees a Total Store Ordering memory ordering [39].

ALGORITHM 1: spanlist_enq

```

1 Function spanlist_enq (list, node)
2   spanlist_t *list;                                ▷ the span-list
3   spannode_t *node;                                ▷ the enqueueing node
4   tail_node ← list->tail;
5   node->next ← NULL;
6   ST_REL(&tail_node->next, node);                  ▷ ensure all store operations commit
7   list->tail ← node;

```

ALGORITHM 2: help_finishing_req

```

1 typedef struct{
2   union{
3     struct{ word_t pending : 1; word_t phase : WORD_BITS -1; }
4     word_t phase_pending; }                                ▷ the last bit of phase_pending is a pending flag
5   word_t last_phase; } helprec_t ;
6 Function help_finishing_req (list, req, sp, pis_null)
7   spanlist_t *list;                                    ▷ the span-list
8   helprec_t *req;                                     ▷ the helping request record
9   span_t **sp;                                         ▷ the pointer of the span temporarily held by the thread
10  bool_t *pis_null;                                    ▷ the empty flag of a span list
11  s ← *sp;
12  phase_pending ← req->phase_pending; phase ← get_phase(phase_pending);
13  while is_pending(phase_pending) && get_phase(phase_pending) == phase do
14    if s == NULL then
15      s ← try_lockfree_pop_head(list, pis_null);        ▷ lock-free attempt
16      if *pis_null then
17        break;
18    else
19      // try finishing the request with the temporarily held span s
20      if CAS(&req->phase_pending, &phase_pending, s) then
21        s ← NULL; break;                                ▷ the temporarily held span s is consumed
22      // if the CAS operation fails, then the phase_pending should have changed
23  *sp ← s;                                              ▷ keep the span

```

As shown in Algorithm 2, line 15, the lock-free dequeuing procedure of the SPMC span-list is the same as the MS-queue [29]. Threads try to change the list head to the next node of the head node via a CAS operation. During the procedure, an ABA problem may occur. For example, the head node read by two threads is the same, yet its next node is already completely different,

leading to errors in dequeuing. We implement the lock-free queuing procedure via LL/SC atomic primitives to prevent this ABA problem. On x86 machines where LL/SC atomic primitives are unsupported, we use simulated LL/SC implemented by the versioned two-word CAS. The LL/SC semantics guarantee that the head node and the successor of the head node read between LL to the successful SC has not been modified. Otherwise, the SC will fail. So LL/SC are immune to ABA problems. There may be thread starvation in the lock-free dequeuing procedure, and in this article, we adopt a simplified KP-queue helping protocol to prevent starvation. In the original KP-queue helping protocol, each thread publishes its request and then helps a finite number of pending threads to make others progress. Once the helping procedure completes, this thread tries to finish its published request. With the helping mechanism, all allocating threads cannot starve. If a thread's lock-free attempt fails, then it indicates that another thread will make progress. The published requests will finally complete after a bounded number of failed lock-free dequeuing. The KP-queue helping protocol ensures that the dequeuing execution steps are $O(N^2)$ bounded. We will discuss the bounded execution time in Section 3.4.

We do not use KP-queue straightforwardly because of the memory reclamation issue. When a thread tries to complete a request in KP-queue, it will allocate a new request record, fill in the node pointer, and replace the original request record with the new one via a CAS operation. It relies on garbage collecting or other memory reclamation mechanisms to avoid ABA problems, leading to performance degradation. This article uses a simplified helping protocol that does not introduce any memory reclamation mechanism. We do not ensure linearizability of the span-list and use the phase counter as a versioning field to avoid the ABA problems when dequeuing. As a result of non-linearizability, a single dequeue request may dequeue two nodes, one held by the request thread itself and another held in the thread request record. This scenario may occur if a thread succeeds in dequeuing a node from the span-list when others have completed the request of this thread. This thread will return the node it dequeued and remain another span in its request record for the next allocation. The span-list is non-linearizable, for it violates the correct concurrent history of a queue that a single dequeue request can only lead to one node being dequeued. However, it is acceptable for an allocator design. In the worst-case scenario, all threads have one unreclaimed span in each helping request record, and the additional memory footprint is bounded. We will discuss the increasing memory footprint in Section 3.5.

Since the helping protocol is a complicated concurrent algorithm, we describe it using pseudocode. The detailed implementation is shown in Algorithm 2, 3, and 4. The execution steps outline as follows.

- (1) In Algorithm 3, lines 8 to 10, threads check if there is any span in its request record that is available for reuse. If so, then threads reclaim the span, clear the request record, and return.
- (2) In Algorithm 3, lines 11 to 12, threads fill in the empty request record with the new phase number and the pending flag bit. The request record data structure is shown in Algorithm 2, lines 1 to 5.
- (3) In Algorithm 3, lines 13 to 24, threads try to help finish a finite number of helping requests.
- (4) The helping routine is shown in Algorithm 2. Threads try to dequeue a node in lock-free manners at line 15 and then fill the node into a pending request record. They clear the pending state via a CAS operation at line 20.
- (5) In Algorithm 4, lines 3 to 14, threads complete their own lock-free dequeuing operations. At the beginning of each lock-free attempt, a thread checks whether their published request finishes with the help of other threads. If so, then the thread returns the span in the completed request record. If a thread succeeds in dequeuing a node from a span-list when other threads have completed the pending request, then it will return the node itself dequeued and remain

another span in its request record for the next allocation. If not, then the pending request will be cleared at line 16.

In steps (4) and (5), if the current SPMC span-list is empty, then the thread will move its traversal pointer to the SPMC span-list of the next heap. If this thread has already traversed the SPMC span-list of P threads, then it clears its pending request and returns NULL. These threads will mark the current position during the traversal to avoid additional overhead next time.

ALGORITHM 3: spanlists_acquire_span (part 1)

```

1  helprec_t hreqs[N][C];           ▷ global helping request records where N is the number of threads
2  Function spanlists_acquire_span (heap, szcls, tid)
3      heap_t *heap;                  ▷ the heap pointer
4      cls_t szcls;                   ▷ the size class of the requested span
5      uint_t tid;                    ▷ the current thread identifier
6      heap_q ← heap->cur_query[szcls]; hpos ← heap->helping_pos[szcls];
7      my_req ← &hreqs[tid][szcls];
8      s ← reclaim_request(my_req);   ▷ reclaim the request record using a SWAP primitive
9      if s ≠ NULL then
10         return s;
11     phase_pending = ((my_req->last_phase++ < 1) | TRUE);   ▷ publish a span request
12     my_req->phase_pending = phase_pending;
13     while help_count < H && help_query < P do
14         help_tid ← (hpos + help_count) % N;               ▷ the helping procedure
15         req ← &hreq[help_tid][szcls];
16         help_phase_pending ← req->phase_pending;
17         if is_pending(help_phase_pending) then
18             help_finishing_req(&heap_q->spanlist[szcls], req, &s, &list_is_null);
19             if list_is_null then
20                 help_query++;   ▷ current span-list is empty, move to next span-list
21                 heap_q ← heap_q->next;
22                 list_is_null ← FALSE;
23                 continue;
24             help_count++;       ▷ help to finish the request until it has been completed
25     // try finishing the thread's own request             ▷ to be continued in Algorithm 4
  
```

3.3 Span Allocation and Deallocation

The span header maintains the metadata in two cache lines. The first cache line is used for local free, including local free-list, local free block count, and block size, and the second is used for remote deallocation, containing an MPSC remote free-list and free block count. Span is initialized by dividing the span memory into small memory blocks and pushing them into the local free-list. For reduced initializing overhead, the free-list adopts a new pointer encoding. If the next pointer of the free-list head node is *NEXT_FREE*, then the next neighboring block (*nodepointer* + *blocksize*) is available.

In span allocations, threads first try to get memory from the local free-list in a non-empty span. If this succeeds, then the allocating thread decreases the local free block count. If the local free-list is empty, then the allocating thread sets the span owner to NULL and then commits the local free count to the free block count by an FAA atomic primitive. Figure 5 shows the reclamation of the spans. The allocating thread will discard the span if the value returned by the FAA plus the local

ALGORITHM 4: spanlists_acquire_span (part 2)

```

1 Function spanlists_acquire_span (heap, szcls, tid)
2   // continued from Algorithm 3                                ▶ now finish the thread's own request
3   while help_query < P do
4     phase_pending ← req->phase_pending;
5     if is_pending(phase_pending) then
6       help_finish_req(&heap_q->spanlist[szcls], my_req, &s, &list_is_null);
7       if list_is_null then
8         help_query++;
9         heap_q ← heap_q->next;                                ▶ move to the next span-list and retry
10        list_is_null ← FALSE;
11        continue;
12      else
13        s ← reclaim_request(my_req);                            ▶ reclaim the span if the request completed
14        goto RET;
15      // if no span is acquired, try to clear the pending flag
16      CAS(&my_req->phase_pending, &phase_pending, NULL);
17      RET: heap->cur_query[szcls] ← heap_q;                    ▶ mark the help_tid and the heap_q
18      heap->helping_pos[szcls] ← help_tid;
19      return s;

```

free count is less than or equal to zero. In other words, the free block count will be less than zero after the commit. If not, then the remote free-list is non-empty, and the allocating thread reclaims the span via a SWAP instruction. If free-lists are blocked by a remote deallocating thread where the linking of the node has not finished, then the use of this span will delay. Then the allocating thread will enqueue this span to the tail of the local span-list.

In span deallocations, threads will check the owner of the span where the free memory block is. If the deallocating thread is the span owner, then it will push the free memory blocks into the local free list and then increase the local free block count. If not, then the free memory blocks are pushed into MPSC wait-free remote free-list by the deallocating thread, and then the free block count increase via an FAA atomic primitive. Since the remote MPSC free-list is in the same cache line as the free block count, the remote deallocation requires only one interconnect overhead in most cases. When the value returned by this FAA operations equals zero, it indicates that the span has been discarded and ownerless, and now the deallocating thread acquires its ownership. Then it reclaims the remote free-list of the span and reuses the memory.

In Figure 5, there is a possibility that the actual number of free memory blocks in the remote free-list reclaimed by the span owner at (d) is more than *free_count* when the deallocating threads halt at (a.1) before increasing the *free_count*. Therefore, during the allocation, the span owner thread may find *local_free_count* + *free_count* < 0 at (c). In this case, the owner thread will still discard the span. Until the deallocating thread increases the *free_count* to 1, the span is reclaimed.

In the allocating procedure, the thread first finds a non-empty span to allocate. When there is no span of suitable size, the allocating thread tries to acquire a span from the SPMC wait-free span-lists of others. If it fails to get a span or the free-lists in the span local free-list is blocked, then the allocating thread attempts to request the full span from the SPMC wait-free span-lists of others and initialize it to a specific size class. Then, it tries allocating in acquired span. If all the attempts above fail, then the allocating threads request the new raw span from the operating system via system calls and retry the allocating in this span.

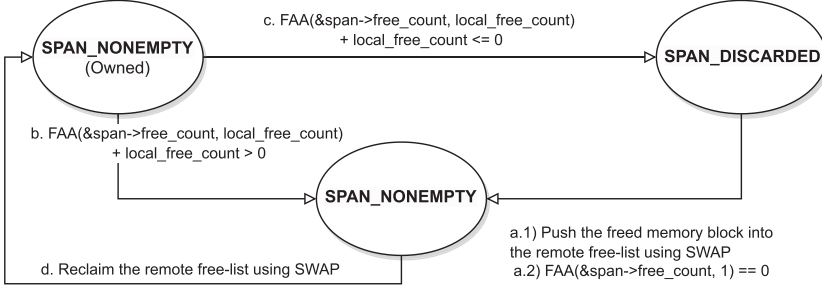


Fig. 5. The reclamation of spans.

The execution steps of the allocation routine are bounded. The only loop during the allocation is acquiring a span from the SPMC span-lists of other threads, where the execution steps are bounded with the helping protocol. Also, the worst-case additional memory footprint is bounded. A parameter K limits the number of private spans in a per-thread heap. Any extra span owned by a thread will be set to ownerless and pushed into an SPMC wait-free span-list.

In the deallocating procedure, the thread will locate the span header through the memory pointer. Since the spans are fixed size and $2^{\log_2 S}$ aligned, the address of a span header can be calculated by masking trailing bits of the memory pointer. If a span becomes the full span during the local deallocation, then the deallocating thread will use *madvise()* to mark the span as MADV_FREE. The deallocation steps are bounded and require no more than three atomic operations.

3.4 Wait-freedom

In this subsection, we will discuss the wait-freedom of wfspan.

LEMMA 3.1. *Procedure try_lockfree_pop_head is $O(1)$ bounded wait-free.*

Proof outline: The implementation of try_lockfree_pop_head is the lock-free dequeuing operation in MS-queue. Since only one attempt is tried, the routine is $O(1)$ bounded wait-free.

LEMMA 3.2. *A published helping request will complete in $O(\frac{N-1}{H} \cdot (N-1) + 1)$ steps.*

Proof outline: The helping procedure uses the fast helping protocol from KP-queue. If a thread T_N cannot progress, then other $N-1$ threads complete the published request from T_N . In the worst case, threads $T_1, T_2, \dots, T_{N-1}$ begin the helping procedure from the thread T_1 , which is the farthest distance away from the thread T_N being helped. These $N-1$ threads help H threads per round. Under the worst thread interleaving, there are at most $\frac{N-1}{H} \cdot (N-1) + 1$ lock-free attempts before a helping request is completed. According to Lemma 3.1, a lock-free attempt is $O(1)$ bounded. So a published helping request is $O(\frac{N-1}{H} \cdot (N-1) + 1)$ bounded wait-free.

THEOREM 3.3. *Procedure spanlists_acquire_span is $O(N^2)$ bounded wait-free.*

Proof outline: In this procedure, the number of helped requests is limited by H . A thread has at most $H+1$ published requests to complete, where the extra one is its own request. According to Lemma 3.2, this procedure returns after at most $\frac{N-1}{H} \cdot (N-1) \cdot (H+1) + P-1$ try_lockfree_pop_head attempts, which is $O(N^2)$ bounded.

The deallocating routine has no loops and is $O(1)$ bounded wait-free. The allocating routine is limited by spanlists_acquire_span, which is $O(N^2)$ bounded wait-free. $O(N^2)$ worst-case execution steps may be more significant on larger multi-core systems. However, by replacing the helping

protocol used in `spanlists_acquire_span`, we can achieve $O(N)$ worst-case execution steps. For example, the helping protocol in `wfqueue` [18] shares the global maximum phase, and threads will help all requests whose phase number is less than the global phase. This helping protocol guarantees $O(c + N)$ bounded wait-free. However, a global synchronization overhead c may decrease average performance. We will evaluate these two different helping protocols in the evaluations.

One thing to note is that it is not strictly lock-free on some ISAs [33], since the weak LL/SC is used in `try_lockfree_pop_head`. The weak LL/SC means that the lock-free attempt can keep being interrupted before the SC completes, causing the SC to fail forever. This problem can also occur in other lock-free allocators that use weak LL/SC. Although it is unlikely to happen in practical systems, it may prevent the system from making progress. In this case, the lock-free progress guarantee requires that at least one thread cannot be interrupted and complete the LL/SC procedure. Otherwise, strong LL/SC immune to interrupts is needed. On the x86/64 ISA, the emulated LL/SC is strong LL/SC implemented via versioning CAS2. In this case, the program can still progress even if LL/SC is interrupted at every instruction between LL and SC. On the aarch64 ISA, since only weak LL/SC is supported, some workarounds can address the problem. One-word versioning CAS can emulate the strong LL/SC. Since the valid physical address on aarch64 is only 48 bits, and the trailing bits of the span pointer are also useless, we can use the $16 + \log_2 S$ bits to encode the version.

3.5 Memory Footprint

In this subsection, we discuss the worst-case memory footprint of our implementation.

LEMMA 3.4. *All threads can produce at most $N \cdot C \cdot S$ additional memory footprint in their helping request records.*

Proof outline: Due to the non-linearizable wait-free span acquiring, a requesting thread T_i can acquire up to two spans, one currently held by it, and the other is in the helping request record. However, this is acceptable for the allocator design, and the span left in the helping request record will be reclaimed at the next span acquiring procedure. In the worst case, each thread acquires an additional span, and these spans are in the helping request records. Considering $N \cdot C$ helping request records exist, there are at most $N \cdot C \cdot S$ additional memory footprint.

LEMMA 3.5. *Span-lists querying in `spanlists_acquire_span` can cause at most $(\lceil N/P \rceil - 1) \cdot (N - 1) \cdot C \cdot S$ additional memory footprint.*

Proof outline: Consider a worst-case scenario where all available spans are in span-list of T_n . Other threads should traverse starting from T_1 . They need $\lceil N/P \rceil - 1$ rounds until they find a non-empty span-list. Each failed `spanlists_acquire_span` may lead to a span allocation from the upper layer. Considering all $N - 1$ threads and C size classes, span-lists querying in `spanlists_acquire_span` will lead to at most $(\lceil N/P \rceil - 1) \cdot (N - 1) \cdot C \cdot S$ additional memory footprint. When $P = N$, this memory overhead can reduce to zero.

LEMMA 3.6. *The use of MPSC free-list can result in at most $N \cdot (N - 1) \cdot C \cdot S$ additional memory footprint.*

Proof outline: The span with the blocked free-list will be pushed into the thread-local span-list for delayed use. In the worst case, there is only one request thread, and the other $N - 1$ remote free-threads halts before the next free memory blocks are linked. Thus, $N - 1$ spans are unavailable and pushed to the local span-list. In the worst case of thread interleaving, each thread will encounter the scenario for each size class, which results in an additional memory footprint of $N \cdot (N - 1) \cdot C \cdot S$.

Wfspan achieves wait-freedom at the cost of increasing but bounded memory footprint. The most additional memory footprint $A(N)$ caused by the design of wfspan is the summary of three

Table 2. The Specifications of the Machines

Platform	OS	CPU	RAM
x86/64 Zen 2	Linux 5.14.8/Fedora 35	AMD Ryzen-3700x (SMT enabled) 16× cores at 4.2 Ghz	16GB DDR4-3800
aarch64 Kyro 680	Linux 5.4.61/Android 11 with Ubuntu 18.04 in docker	Qualcomm Snapdragon 888 1 × 2.8 GHz, 3 × 2.4 GHz, 4 × 1.8 GHz	8GB LPDDR5-3200

Table 3. The Versions of the Compared Allocators

Allocator	Version
glibc	2.34 (on x86)/2.27 (on aarch64)
tc	gerpftools-2.8.1
je	5.2.1
hd	a43ac40
mi	v2.0.0
sn	0.5.3
wfspan	0.0.1

parts in Lemma 3.4, 3.5, and 3.6, as shown in Equation (1),

$$A(N) = (N + (\lceil N/P \rceil + N - 1) \cdot (N - 1)) \cdot C \cdot S. \quad (1)$$

From Equation (1), the allocator design of wfspan may lead to $O(N^2)$ memory footprint growth. We can construct such a scenario to measure the worst-case memory footprint. The thread T_1 continuously allocates memory blocks of total size $m+k$ and then free them, where k is the number of allocating rounds. The free spans will enqueue the SPMC span-list of T_1 . Then other threads allocate memory of total size $m+k$ and deallocate them. They will consume all memory in the SPMC span-list of T_1 . The spans will frequently move between all threads and T_1 . This allocation pattern has the increasing requested memory size and may lead to a significant increase in memory footprint of size $k \cdot (\lceil N/P \rceil - 1) \cdot (N - 1) \cdot C \cdot S$. We will evaluate this allocation pattern in the evaluation section.

4 EVALUATION

To evaluate wfspan in terms of average performance/memory footprint, worst-case execution time/worst-case memory footprint, and runtime memory utilization, we ran memory allocator benchmarks on a 16-core x86 machine and an 8-core heterogeneous aarch64 machine. The wfspan and the wfspan with wfqueue helping protocol (wfqp) compared with six cross-platform practical allocators. As shown in Table 2, these allocators included Glibc malloc, mimalloc, tcmalloc, jemalloc, snmalloc, and hd (hoard), where Glibc, tcmalloc, jemalloc, and hoard are lock-based allocators, and mimalloc and snmalloc are lock-free allocators. We ran all benchmarks on two different machines. The specifications of these machines are shown in Table 3.

The wfspan was compiled as a dynamic library using GCC with the `-O3` optimization flag. The version of GCC is 11.2.1 on the x86 machine and 7.3.0 on the aarch64 machine. On the x86 machine, we used the `-mcx16` flag to enable CAS2. On the aarch64 machine, we used the `-march=armv8a-lse` flag to ensure that FAA, SWAP, CAS atomic primitives were correctly compiled. For the allocator parameter settings, we defined $SPAN_SIZE = 64kBytes$, $K = 40$, $H = 1$, $P = N$, where N is number of requesting threads.

4.1 Average Performance and Memory Footprint

We ran micro-benchmarks and real application workloads to evaluate the average performance and memory footprint. Most of the test workloads were from the mimalloc-bench [22]. Some single-threaded variants of the workloads from the mimalloc-bench were not included as they are not representative. We used seven malloc implementations mentioned before for comparison. The malloc libraries were loaded via `LD_PRELOAD`, and we measured the total elapsed time and max **Resident Set Size (RSS)** using the `/usr/bin/time -v` command. In this section, we use RSS to represent the memory footprint. The RSS includes the memory requested by the user, the metadata size of memory allocator implementations, and process-related data.

The micro-benchmarks includes:

- **xmalloc**: A benchmark from Lever and Boreham [24], where massive allocating threads and deallocating threads are created to complete workloads with the asymmetric allocating pattern. This benchmark is unfriendly to the allocators with the thread-local allocation buffer.
- **cscratch**: A cache stressed benchmark from hoard [4], mainly for detecting the passive cache false-sharing, where threads allocate small memory objects and give them to other threads to access.
- **rp**: A benchmark from rpmalloc [16] where all threads allocate memory with an exponent size distribution from 8 to 16,000 bytes. In this benchmark, every two loops of deallocation are remote.
- **sh6, sh8**: Benchmarks from smartheap [30], where threads may free part of allocated memory in reverse order.
- **mstress**: A memory stress workload from mimalloc-bench [22], aiming at simulating real-world server-like allocation patterns.
- **alloc-test1, alloc-testN**: Memory allocation workloads with the Pareto size distribution, where N is the maximum number of hardware threads [1].

The practical application workloads include:

- **cfrac**: A single-threaded computing workload that calculates the factorization of integers, where many short-lived small objects are allocated and freed.
- **barnes**: A scientific computing application from SPLASH2 [43] benchmark suite, simulating the gravity of 163,840 particles.
- **espresso**: A logic array analyzer workload from Grunwald et al. [11].
- **z3**: An **Satisfiability Modulo Theories (SMT)** solver from Microsoft Research [6].
- **redis**: A Redis workload where each thread pushes ten nodes and then pops them in 1 million requests.
- **cxqueue**: Concurrent data structures workload from CX [5], using different lock-free and wait-free queues, where each thread enqueues 16,384 nodes and then dequeues them. In this workload, MSqueue, wfqueue, and CRT-queue are used, and each enqueueing operation is accompanied by a malloc request of a new node. Each dequeuing operation may trigger the deallocation of retired nodes.
- **larson**: A benchmark from Larson et al. [21], simulating a remote deallocation pattern observed in real-world servers.
- **lean**: A compiler workload from mimalloc [23]. In this workload, Lean compiler is used to compile its standard library, where intensive allocations and deallocations of dynamic objects exist.

On the x86 machine and the aarch64 machine, we ran these workloads eight times with different malloc libraries loaded. The average results and standard deviation are presented in Figures 6, 7,

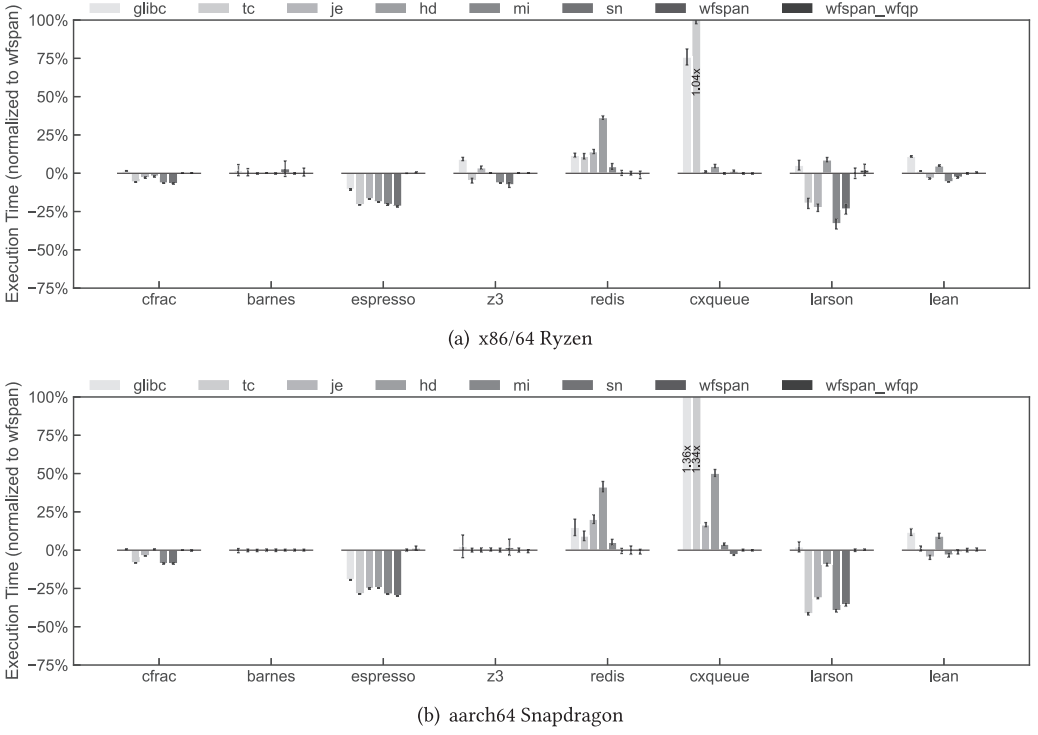


Fig. 6. The execution time of real application workloads.

8, and 9. Figures 6 and 8 show the execution time of these workloads, where the x -axis is the workloads, and the y -axis is the execution time normalized to wfspan. The lower execution time represents the better average performance. Figure 7 and 9 show the max RSS, where the y -axis is the max RSS normalized to wfspan. The lower value of max RSS represents the lower memory footprint. The geometric mean of results is as shown in Table 4.

According to Table 4, wfspan achieves a low memory footprint and competitive performance on both test machines. Both the wfspan implementations have significant performance advantages (up to 49%) over lock-based allocators. Compared to the state-of-the-art lock-free allocators, wfspan shows a slightly less performance (about 8–13%) but lower RSS (about 7–14%). The decentralized architecture of wfspan does not lead to much higher RSS in these workloads, which is encouraging.

The experimental results show that wfspan has worse performance on single-threaded workloads (cfrac, barnes, espresso, z3) due to the more complicated allocating procedure. However, the simplified allocator design helps reduce metadata memory footprint, and thus wfspan shows less RSS in these workloads. In most parallel workloads, wfspan achieves competitive or better performance than other allocators, except for larsen, sh6, and sh8. In these workloads where massive remote deallocations exist, wfspan shows slightly lower performance, since it does not employ a cache mechanism for remote freeing. Each remote deallocation requires at least one atomic operation, leading to increased execution time. In two alloc-test workloads where massive local allocations exist, wfspan has the lower performance, because the thread-local span limitation K will lead to extra spans pushed into the SPMC span-lists, which results in more remote deallocations with more execution time overhead.

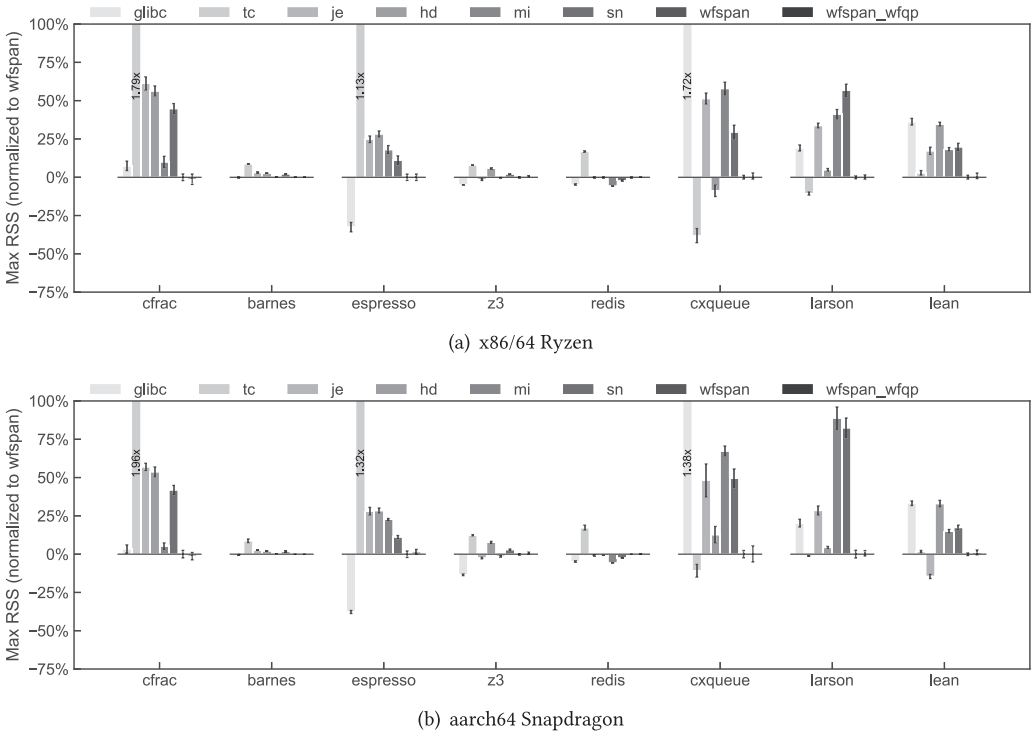


Fig. 7. The max RSS of real application workloads.

Table 4. The Geometric Mean of Normalized Execution Time and Max RSS

machine		glibc	tc	je	hd	mi	sn	wfspan	wfq
x86/64 Ryzen	time	1.359	1.490	1.172	1.166	0.869	0.889	1.000	1.008
	max RSS	1.069	1.089	1.072	1.379	1.069	1.104	1.000	1.039
aarch64 Snapdragon	time	1.420	1.293	1.198	1.212	0.918	0.906	1.000	1.004
	max RSS	0.990	1.170	1.043	1.342	1.145	1.137	1.000	1.031

Table 5. Max RSS in the WCET Microbenchmark

Allocators	Max RSS (kBytes)	
	x86/64 Ryzen	aarch64 Snapdragon
glibc	423750 ± 7071	249663 ± 11825
tc	496139 ± 7441	385541 ± 4663
je	409813 ± 7659	277046 ± 7271
hd	551694 ± 672	417060 ± 3201
mi	396357 ± 6370	218832 ± 9371
sn	346019 ± 5707	199854 ± 8289
wfspan	461247 ± 4410	340830 ± 24042
wfspan_wfq	472515 ± 7521	347737 ± 22100

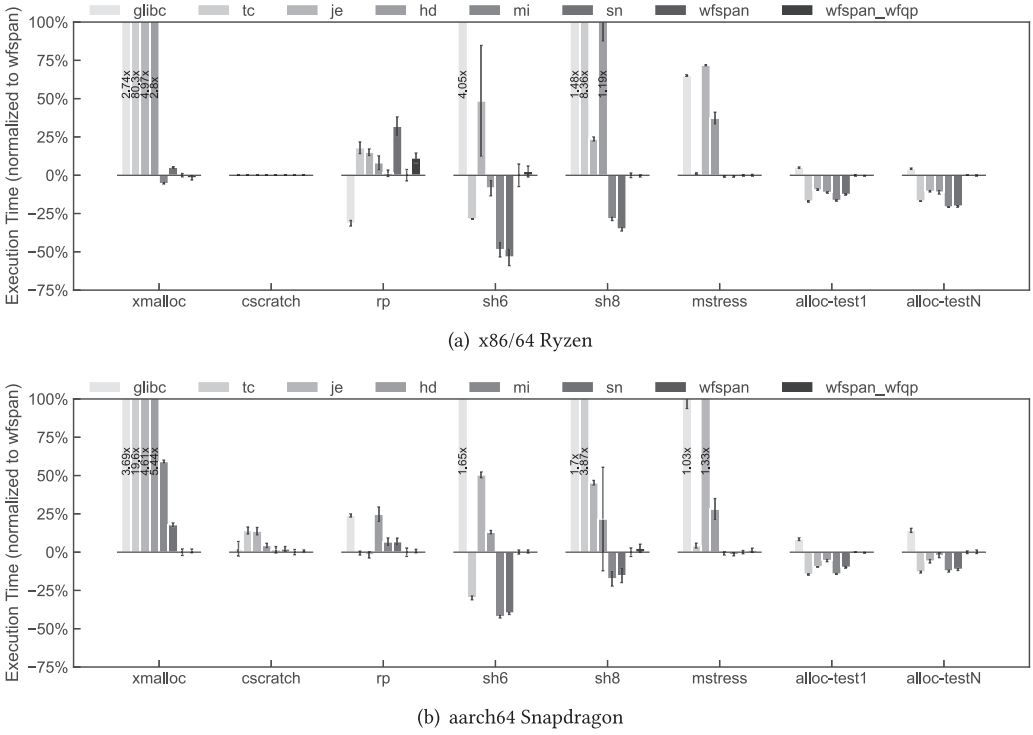


Fig. 8. The execution time of micro-benchmark workloads.

Table 6. Maximum RSS and Heap Size in the Worst-case Memory Footprint Microbenchmark

Allocators	Max RSS (kBytes)	Max Heap Size (kBytes)
glibc	308682 $\pm$ 21907	/
tc	243074 $\pm$ 713	/
je	1325669 $\pm$ 1068	/
hd	998155 $\pm$ 204	/
mi	346474 $\pm$ 10130	/
sn	314390 $\pm$ 5228	/
wfspace	416216 $\pm$ 21835	412122 $\pm$ 21699
wfspace_half	514842 $\pm$ 14419	510682 $\pm$ 14409
wfspace_quater	731230 $\pm$ 25625	727219 $\pm$ 25594

4.2 Worst-case execution time and memory footprint

The Linux kernel is not a real-time OS kernel, and the workloads run with context switches, interrupts, and thread migration. So it is impossible to measure the worst-case execution time accurately for each memory allocator. Here we use a statistical worst-case execution time measurement, the percentile of memory request latency.

The worst-case execution time of a memory allocator can be observed when there is most-heavily contention among threads. Nevertheless, there is no such allocation scenario in real-world applications. Even the maximum dynamic allocating rate in wait-free or lock-free queues is several

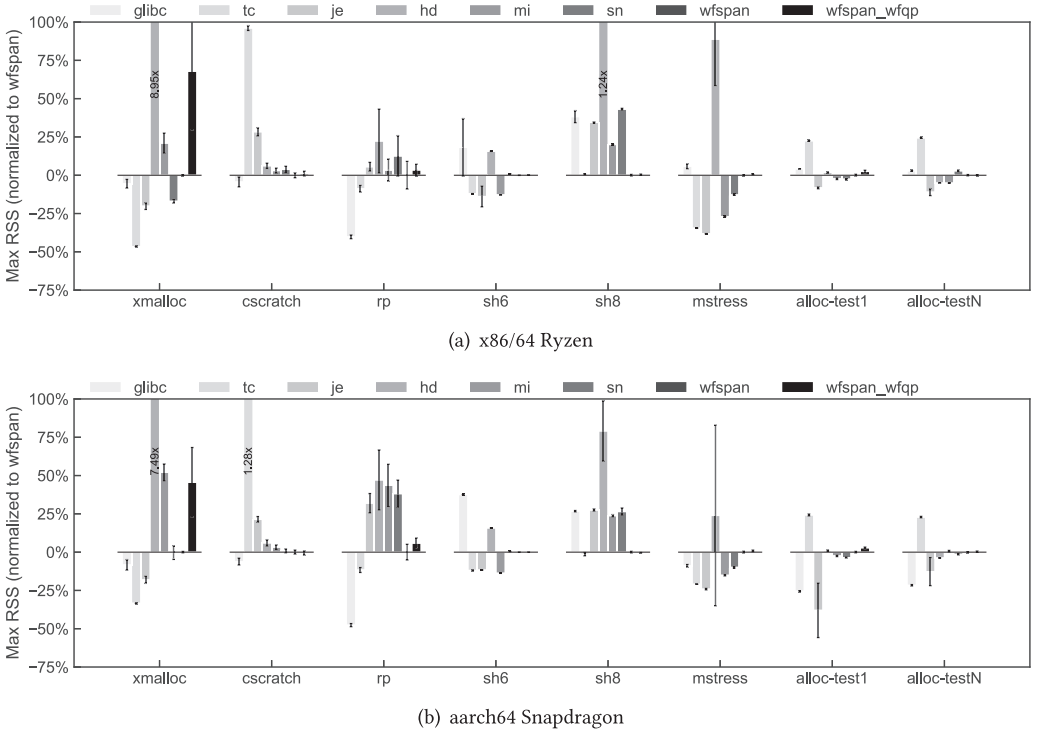


Fig. 9. The max RSS of micro-benchmark workloads.

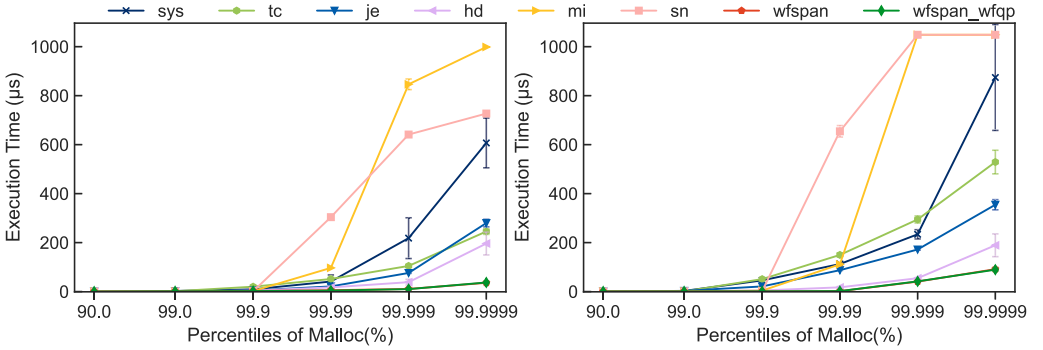


Fig. 10. The percentiles of memory allocating latency (left: x86/64 Ryzen, right: aarch64 Snapdragon).

million requests per second. Most memory requests are satisfied within the thread-local allocation buffer, which does not cause synchronization. To emulate the worst-case memory requests and measure the execution time, we implemented a stressed concurrent workload where all threads complete 100M allocations and deallocations of constant size in multiple batches. To exhaust the thread-local allocation buffer, we design an asymmetric allocating pattern. It means each thread allocates a different number of objects for each round. All allocated objects in a batch are shuffled and published to a global list, and later they will be freed by other threads to ensure most deallocations are remote.

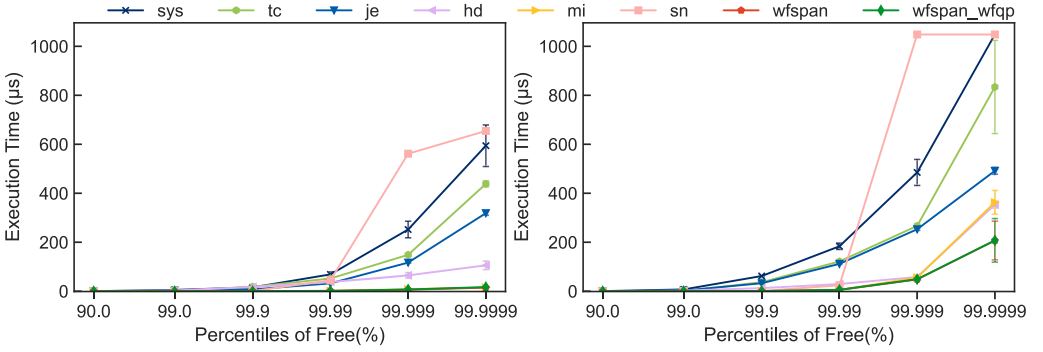


Fig. 11. The percentiles of memory deallocating latency (left: x86/64 Ryzen, right: aarch64 Snapdragon).

Measurement Methodology. We use hardware instructions to read the CPU cycles to measure the latency of memory requests. The measurement may cause a memory barrier that affects a memory allocator’s performance. To reduce the measuring overhead, we sample the CPU cycles of every eight memory requests. Also, to simplify the storage of results when measuring, we employ a fixed-size array to count the CPU cycle in a specific interval, so there is an upper bound for the recorded CPU cycle. If a measured CPU cycle is larger than the upper bound, then it will be counted as the upper bound.

We ran the worst-case execution time workload 10 times, with a constant object size from 16 bytes to 2,048 bytes. We compared different malloc dynamic libraries in this workload. The percentiles of memory request latency and their standard deviations were calculated as shown in Figure 10 and 11. In these figures, the x -axis is the percentile of memory requests, and the y -axis is the latency of memory requests, converted from CPU cycles to microseconds. The max RSS in this micro-benchmark is shown in Table 5.

The results show that the latency of memory requests for our implementations, including wfspan and wfspan-wfq, is relatively stable and lower than compared memory allocators on both aarch64 and x86 machines. Especially in terms of allocation, wfspan showed significantly lower latency due to its wait-free span acquiring, without increasing much memory footprint. However, this latency on the aarch64 machine is slightly higher. Because the Android OS has many system threads running in the background with the energy-aware scheduling enabled, where interrupts and schedulings are more frequent than on the x86 machine, resulting in higher standard deviations and latency.

Among the lock-based allocators, hoard shows the lowest latency. In lock-free allocators, mimalloc has the same latency as the wfspan, because they have a similar $O(1)$ deallocating pattern. Also, we can see that sn and mi have even higher allocation latency than these lock-based allocators, because they have some allocation patterns that manage memory in batches. These patterns can improve average performance at the cost of a higher worst-case execution time. Besides, the difference between wfspan with $O(N^2)$ helping protocol and wfspan-wfq with $O(N)$ helping protocol is negligible for this number of threads. The wfspan-wfq shows a slightly lower latency than the wfspan but a worse average performance.

To evaluate the worst-case memory footprint of the wfspan, we designed an allocation workload, wcmf_test, in which the requested memory size grows with the number of allocation round k , which is discussed in Section 3.5. We use a random size test function with a linear distribution of 16,384 bytes ($k = 100M$) and a series of fixed-size test functions of 16-4096 ($k = 2M$) in this allocation pattern. One selected thread will collect all spans from others in this allocating pattern.

And then others will allocate memory parallelly. If the other threads fail to find that thread holding all spans in P -traversals, then they will require an additional memory footprint. The number of memory requests increases after each round of span transfer is complete, thus triggering new spans acquiring in the next span acquisition round. We can observe the contribution of the P to the growth of RSS by changing the parameter P . Here we use $P = N$ (wfspan), $P = N/2$ (wfspan_half), and $P = N/4$ (wfspan_quater) for comparison. Considering that the process had additional memory occupation that might distort the results, we also recorded the maximum heap size of the wfspan for comparison.

The results for five runs on the x86 machine are shown in Table 6. In this micro-benchmark, the max RSS of the compared allocators shows a significant difference. The lock-based allocators jemalloc and hoard have more memory footprint than the other allocators. Also, We observed that when $P = N$, the memory footprint of wfspan is slightly higher than mimalloc and smalloc. When $P < N$, the memory footprint of wfspan is inversely proportional to P , which is significantly higher than mimalloc and smalloc. Also, the results support that our allocator has a reasonable memory footprint in such a stressed workload if we use $P = N$.

4.3 Runtime Memory Utilization

Considering that the maximum RSS may not reflect the actual memory utilization, we will evaluate the runtime memory utilization in this section. We measured the runtime memory utilization in three workloads: cfrac, mstress, and wcmf_test. cfrac is a single-threaded benchmark that can be used for comparing the runtime metadata size of allocators. mstress is a multi-threaded workload that can reflect memory utilization in parallel. wcmf_test is a workload that triggers the worst-case memory footprint of wfspan.

Measurement Methodology. We implement a static library libmemstat that intercepts the malloc(3) calls in C programs. It will create a monitor thread that collects the total malloc/calloc/realloc size and total free size for each thread every 100 ms, queries the current RSS, and records the data. We use libmemstat at link-time. It can replace the malloc(3) implementation with `-Wl, -wrap, malloc` options. This evaluation takes the total malloc/calloc/realloc size minus the total free size as the reserved memory size. Since the monitor thread requires an additional hardware thread, mstress and wcmf_test are executed with the number of threads $N = 15$ to reduce the impacts on the evaluation results.

The evaluation results on the x86 Ryzen machine are shown in Figure 12, where the x -axis is the normalized elapsed time, and the y -axis is the ratio of reserved memory size to RSS. Due to the large variation of y value for some allocators, the results are presented as least square polynomial fitted curves for comparison. The average and maximum ratio of reserved memory size to RSS for five runs are presented in Table 7.

According to the results of cfrac, the maximum reserved memory size of cfrac is only about 10% of the RSS. The ratio of reserved memory size to RSS for three wfspan variants is higher than other memory allocators because of the small metadata size of wfspan. There are slight differences between the three wfspan variants, which may be caused by the execution of the monitor thread.

From the results of mstress, there is little difference between the three wfspan variants. All of them have better memory utilization than hoard. The ratio of reserved memory size to RSS can be greater than 1.0, because the actual used memory size is smaller than the reserved memory size, so the operating system does not allocate physical memory pages for the unused memory pages. In addition, we can observe a large variation in the results of jemalloc and Glibc. Because for jemalloc and Glibc, the workload triggers the path to return memory pages to the operating system.

The effect of parameter P on memory utilization for wfspan can be observed from the evaluation results of wcmf_test. In this workload, the three variants of wfspan show memory utilization

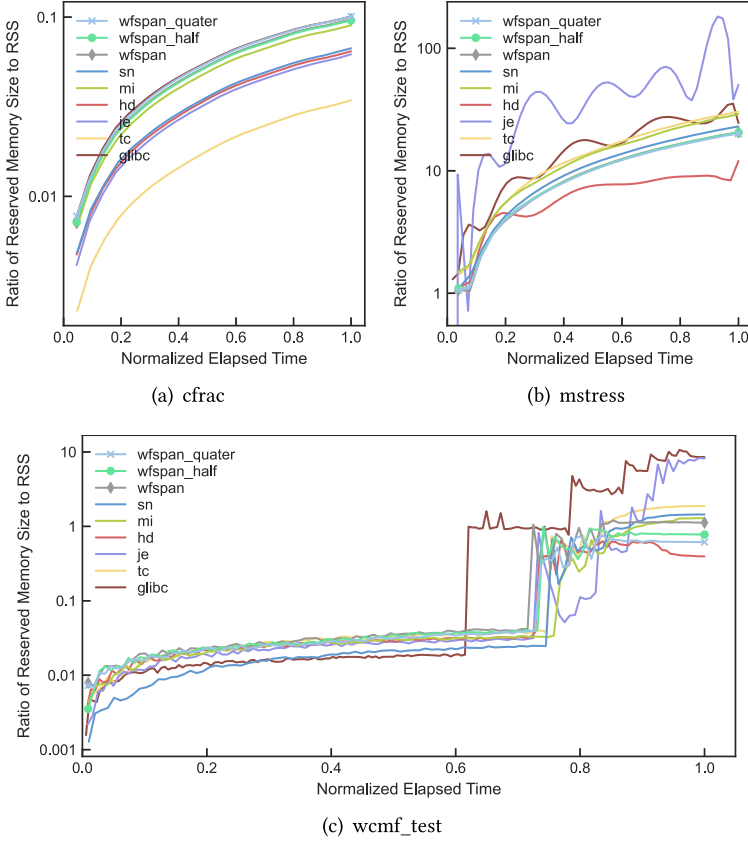


Fig. 12. The runtime memory utilization.

differences. When $P = N$, wfspace achieves the best memory utilization. Besides, Glibc and jemalloc have significantly better memory utilization than the other memory allocators in the later stages of this workload, because they frequently return memory pages to the operating system.

In summary, according to the runtime memory utilization evaluation results, the memory overhead of wfspace is acceptable. Even in the worst-case workload, wfspace_quarter can still achieve better memory utilization than hoard.

5 RELATED WORK

The dynamic memory allocator is a fundamental component of modern operating systems and application programs. There are some researches focusing on improving performance, memory utilization, and worst-case latency of the dynamic memory allocators.

Two-Level Segregated-Fit, an $O(1)$ memory allocation algorithm. **Two-Level Segregated-Fit (TLSF)** [26] is a memory allocation algorithm with $O(1)$ execution time, which uses two-level size classes. The size classes are fast indexed via a Count Leading Zeroes instruction. TLSF employs a boundary-tagged block header to maintain metadata, and blocks need to be split to the required size when allocated and merged when freed. TLSF works well on a single-core processor. However, TLSF should ensure thread safety by a global heap lock for multi-core systems, leading to poor scalability.

Table 7. The Average and Maximum Runtime Memory Utilization

	cfrac		mstress		wcmf_test	
	avg	max	avg	max	avg	max
glibc	0.056	0.099	16.795	45.167	1.528	10.448
	± 0.002	± 0.002	± 0.469	± 4.084	± 0.049	± 0.777
tc	0.019	0.035	15.509	30.520	0.348	1.873
	± 0.000	± 0.000	± 0.244	± 0.079	± 0.040	± 0.007
je	0.034	0.062	44.259	286.842	0.690	8.804
	± 0.001	± 0.002	± 1.931	± 104.914	± 0.058	± 0.618
hd	0.035	0.064	7.835	16.046	0.149	0.677
	± 0.001	± 0.001	± 2.091	± 4.946	± 0.005	± 0.030
mi	0.052	0.093	14.409	28.559	0.239	1.421
	± 0.001	± 0.002	± 0.236	± 0.400	± 0.019	± 0.281
sn	0.037	0.067	11.875	23.098	0.267	1.411
	± 0.001	± 0.001	± 0.033	± 0.080	± 0.013	± 0.033
wfspace	0.056	0.100	10.748	20.852	0.285	1.122
	± 0.001	± 0.002	± 0.035	± 0.134	± 0.012	± 0.075
wfspace_half	0.054	0.097	10.785	20.688	0.239	1.026
	± 0.001	± 0.002	± 0.116	± 0.071	± 0.017	± 0.040
wfspace_quarter	0.055	0.098	10.562	20.299	0.191	0.921
	± 0.001	± 0.002	± 0.042	± 0.105	± 0.016	± 0.117

Lock-based memory allocators. Locks are widely used synchronization techniques. The modern practical memory allocators, such as ptmalloc, jemalloc [8], tcmalloc [10], and hoard [4], use locks for thread safety. Since heavy contention of a single lock can be a bottleneck of the whole system, all these memory allocators try to mitigate this problem through multi-heap design, fine-grained locking, or thread-caching. Ptmalloc is the malloc implementation from the GNU C library, which provides scalability by multi-heap architecture. In this architecture, lock contention spreads across the different arenas (a thread shared data structure). Jemalloc maintains the arenas that are four times the maximum number of cores. Threads are assigned to a set of arenas to reduce the probability of lock contention in the same arena. Tcmalloc from gperf-tools uses a thread-local cache and a global central cache-lists to manage free memory blocks. Memory allocation and deallocation in the global shared cache lists are in batches, reducing the lock contention. Hoard maintains a global shared heap and per-processor heaps to provide scalability. Besides, both hoard and tcmalloc employ fine-grained CAS spin-lock to avoid voluntary scheduling. Deadlocks may occur in these lock-based memory allocators. Also, from a system perspective, blocking synchronization inevitably introduces scheduling, context switching, and thread migrating among cores, leading to increasing and unpredictable execution times. Even if we replace the mutex with a user-space fine-grained lock that does not involve scheduling, the problem still exists. When the number of blocking threads is larger than the number of processors, the user-space fine-grained lock still requires calling voluntary context switching. Otherwise, the blocking threads may not progress.

Lock-free memory allocators. Lock-free memory allocators use lock-free data structures for synchronization. Lfmalloc [28] is a lock-free allocator based on hoard. Supermalloc [20] ensures high performance and simplified lockless implementation by x86 hardware transactional memory instructions. Snmalloc [25] focuses on producer/consumer memory allocation patterns and handles remote memory deallocation in batches using an MPSC wait-free list. Mimalloc [23] employs different in-span free-lists, where MPSC lock-free lists are used for remote deallocation. Both

snmalloc and mimalloc maintain global lock-free Treiber-stacks for global memory management. The lock-free memory allocators are simpler to implement with better performance than their lock-based counterparts. Furthermore, they do not need to call voluntary context switches. As a result, they achieve high throughput with robustness. However, lock-free does not guarantee bounded execution steps. Thread starvation may occur if one slower thread always fails its lock-free attempt.

Wait-free memory allocator. Stellag et al. [40] introduced a helping mechanism to the lock-free buddy allocator, transforming the buddy allocator into a wait-free manner. Compared to the lock-free implementation, the wait-free property guarantees that all threads will complete memory requests in bounded execution steps. However, this wait-free allocator uses the single heap design, which is not scalable.

Memory Isolated Allocators. In multi-core systems with shared memory, cache associative and memory banks can cause significant variations in memory latency. For hard real-time systems, the variation of memory latency may lead to deadline misses. So the memory isolation strategy is commonly adopted in these systems. Cache sets and memory banks are divided by specific bits of memory addresses. The memory isolated allocators manage memory based on these bits of memory addresses, ensuring that allocated memory is from specific cache sets or memory banks, thus reducing time variation of shared memory. Palloc [45] is a bank-aware memory allocator, where memory pages are isolated according to the memory banks they locate. The memory bank isolation strategy decreases the memory latency, since the read and writes operations on different memory banks are parallelizable. CAMA [14] is a cache-aware memory allocator, which manages free memory according to the cache set. The cache isolation strategy avoids the memory sharing the same cache-sets and provides predictable memory latency.

6 CONCLUSION

Dynamic memory allocation plays a vital role in modern application programs. Modern lock-free memory allocators based on hardware atomic primitives usually provide good performance. However, threads may starve in these lock-free implementations, leading to unbounded worst-case execution time that is not allowed in embedded real-time systems. In this article, we present decentralized wait-free dynamic memory management, wfspan. It bases on non-linearizable wait-free lists and employs a helping mechanism to ensure no starvation in the lock-free implementation. From the perspective of design tradeoff, wfspan guarantees bounded execution steps in both allocation and deallocation procedure, at the cost of increasing bounded worst-case memory footprint. The evaluation results of running benchmarks on an x86/64 machine and an aarch64 machine are encouraging. wfspan achieves competitive performance and memory footprint compared to lock-based (Glibc, je, tc, and hd) and lock-free practical memory allocators(mi and sn) while showing significantly superior to other allocators in terms of worst-case execution time.

REFERENCES

- [1] OLogN Technologies AG. 2018. Retrieved from alloc-test. <https://github.com/node-dot-cpp/alloc-test>.
- [2] Andrew W. Appel and David A. Naumann. 2020. Verified sequential malloc/free. In *Proceedings of the ACM SIGPLAN International Symposium on Memory Management*. 48–59.
- [3] ARM. 2020. ARM BIG.LITTLE. Retrieved from <https://www.arm.com/why-arm/technologies/big-little>.
- [4] Emery D. Berger, Kathryn S. McKinley, Robert D. Blumofe, and Paul R. Wilson. 2000. Hoard: A scalable memory allocator for multithreaded applications. In *Proceedings of the 9th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS IX)*. Association for Computing Machinery, New York, NY, 117–128. <https://doi.org/10.1145/378993.379232>
- [5] Andreia Correia, Pedro Ramalhete, and Pascal Felber. 2020. A wait-free universal construction for large objects. In *Proceedings of the 25th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP'20)*. Association for Computing Machinery, New York, NY, 102–116. <https://doi.org/10.1145/3332466.3374523>

- [6] Leonardo de Moura and Nikolaj Bjørner. 2008. Z3: An efficient SMT solver. In *Tools and Algorithms for the Construction and Analysis of Systems*, C. R. Ramakrishnan and Jakob Rehof (Eds.). Springer, Berlin, 337–340.
- [7] Dave Dice and Alex Garthwaite. 2002. Mostly lock-free malloc. In *Proceedings of the 3rd International Symposium on Memory Management (ISMM'02)*. Association for Computing Machinery, New York, NY, 163–174. <https://doi.org/10.1145/512429.512451>
- [8] Jason Evans. 2006. A Scalable Concurrent malloc(3) Implementation for FreeBSD. Retrieved from <https://www.bsdcn.org/2006/papers/jemalloc.pdf>.
- [9] Panagioti Fatourou and Nikolaos D. Kallimanis. 2014. Highly-efficient wait-free synchronization. *Theor. Comput. Syst.* 55, 3 (October 2014), 475–520. <https://doi.org/10.1007/s00224-013-9491-y>
- [10] Google. 2021. TCMalloc: Thread-Caching Malloc. Retrieved from <https://google.github.io/tcmalloc/design.html>.
- [11] Dirk Grunwald, Benjamin Zorn, and Robert Henderson. 1993. Improving the cache locality of memory allocation. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI'93)*. Association for Computing Machinery, New York, NY, 177–186. <https://doi.org/10.1145/155090.155107>
- [12] Maurice Herlihy. 1991. Wait-free synchronization. *ACM Trans. Program. Lang. Syst.* 13, 1 (Jan. 1991), 124–149. <https://doi.org/10.1145/114005.102808>
- [13] M. Herlihy, V. Luchangco, and M. Moir. 2003. Obstruction-free synchronization: Double-ended queues as an example. In *Proceedings of the 23rd International Conference on Distributed Computing Systems*. 522–529. <https://doi.org/10.1109/ICDCS.2003.1203503>
- [14] J. Herter, P. Backes, F. Hauptenthal, and J. Reineke. 2011. CAMA: A predictable cache-aware memory allocator. In *Proceedings of the 23rd Euromicro Conference on Real-Time Systems*. 23–32.
- [15] Intel. 2021. New Intel Core Processors with Intel Hybrid Technology. Retrieved from <https://www.intel.com/content/www/us/en/products/docs/processors/core/core-processors-with-hybrid-technology-brief.html>.
- [16] Mattias Jansson. 2018. Rampant Pixels Memory Allocator Benchmark. Retrieved from <https://github.com/mjansson/rpmmalloc-benchmark>.
- [17] Andi Kleen. 2013. Lock Elision in the GNU C Library. Retrieved from <https://lwn.net/Articles/534758/>.
- [18] Alex Kogan and Erez Petrank. 2011. Wait-free queues with multiple enqueueers and dequeuers. In *Proceedings of the 16th ACM Symposium on Principles and Practice of Parallel Programming (PPoPP'11)*. Association for Computing Machinery, New York, NY, 223–234. <https://doi.org/10.1145/1941553.1941585>
- [19] Alex Kogan and Erez Petrank. 2012. A methodology for creating fast wait-free data structures. In *Proceedings of the 17th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP'12)*. Association for Computing Machinery, New York, NY, 141–150. <https://doi.org/10.1145/2145816.2145835>
- [20] Bradley C. Kuszmaul. 2015. SuperMalloc: A super fast multithreaded malloc for 64-bit machines. In *Proceedings of the 2015 International Symposium on Memory Management (ISMM'15)*. Association for Computing Machinery, New York, NY, 41–55. <https://doi.org/10.1145/2754169.2754178>
- [21] Per-Åke Larson and Murali Krishnan. 1998. Memory allocation for long-running server applications. In *Proceedings of the 1st International Symposium on Memory Management (ISMM'98)*. Association for Computing Machinery, New York, NY, 176–185. <https://doi.org/10.1145/286860.286880>
- [22] Daan Leijen. 2021. mimalloc-bench. Retrieved from <https://github.com/daanx/mimalloc-bench>.
- [23] Daan Leijen, Benjamin Zorn, and Leonardo de Moura. 2019. Mimalloc: Free list sharding in action. In *Programming Languages and Systems*, Anthony Widjaja Lin (Ed.). Springer International Publishing, Cham, 244–265.
- [24] Chuck Lever and David Boreham. 2000. Malloc() Performance in a Multithreaded Linux Environment. In *Proceedings of the Annual Conference on USENIX Annual Technical Conference (San Diego, California) (ATEC'00)*. USENIX Association, USA, 301–311.
- [25] Paul Liétar, Theodore Butler, Sylvan Clebsch, Sophia Drossopoulou, Juliana Franco, Matthew J. Parkinson, Alex Shamis, Christoph M. Wintersteiger, and David Chisnall. 2019. Smmalloc: A message passing allocator. In *Proceedings of the ACM SIGPLAN International Symposium on Memory Management (ISMM'19)*. Association for Computing Machinery, New York, NY, 122–135. <https://doi.org/10.1145/3315573.3329980>
- [26] M. Masmano, I. Ripoll, A. Crespo, and J. Real. 2004. TLSF: A new dynamic memory allocator for real-time systems. In *Proceedings of the 16th Euromicro Conference on Real-Time Systems (ECRTS'04)*. IEEE Computer Society, USA, 79–86.
- [27] Maged M. Michael. 2004. Hazard pointers: Safe memory reclamation for lock-free objects. *IEEE Trans. Parallel Distrib. Syst.* 15, 6 (June 2004), 491–504. <https://doi.org/10.1109/TPDS.2004.8>
- [28] Maged M. Michael. 2004. Scalable lock-free dynamic memory allocation. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI'04)*. Association for Computing Machinery, New York, NY, 35–46. <https://doi.org/10.1145/996841.996848>
- [29] Maged M. Michael and Michael L. Scott. 1996. Simple, fast, and practical non-blocking and blocking concurrent queue algorithms. In *Proceedings of the 15th Annual ACM Symposium on Principles of Distributed Computing (PODC'96)*. Association for Computing Machinery, New York, NY, 267–275. <https://doi.org/10.1145/248052.248106>

- [30] microquill. 2007. Smart Heap. Retrieved from http://www.microquill.com/smartheap/sh_tspec.htm.
- [31] Ruslan Nikolaev and Binoy Ravindran. 2020. Universal wait-free memory reclamation. In *Proceedings of the 25th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP'20)*. Association for Computing Machinery, New York, NY, 130–143. <https://doi.org/10.1145/3332466.3374540>
- [32] Yaqiong Peng and Zhiyu Hao. 2018. FA-Stack: A fast array-based stack with wait-free progress guarantee. *IEEE Trans. Parallel Distrib. Syst.* 29, 4 (2018), 843–857. <https://doi.org/10.1109/TPDS.2017.2770121>
- [33] Erez Petrank, Madanlal Musuvathi, and Bjarne Steensgaard. 2009. Progress guarantee for parallel programs via bounded lock-freedom. In *Proceedings of the 30th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI'09)*. Association for Computing Machinery, New York, NY, 144–154. <https://doi.org/10.1145/1542476.1542493>
- [34] Isabelle Puaut. 2002. Real-time performance of dynamic memory allocation algorithms. In *Proceedings of the 14th Euromicro Conference on Real-Time Systems (Euromicro RTS'02)*. IEEE, 41–49.
- [35] Ravi Rajwar and James R. Goodman. 2001. Speculative lock elision: Enabling highly concurrent multithreaded execution. In *Proceedings of the 34th Annual ACM/IEEE International Symposium on Microarchitecture (MICRO'34)*. IEEE Computer Society, 294–305.
- [36] Pedro Ramalhete and Andreia Correia. 2017. POSTER: A wait-free queue with wait-free memory reclamation. In *Proceedings of the 22nd ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP'17)*. Association for Computing Machinery, New York, NY, 453–454. <https://doi.org/10.1145/3018743.3019022>
- [37] Sepideh Roghanchi, Jakob Eriksson, and Nilanjana Basu. 2017. Ffw: Delegation is (much) faster than you think. In *Proceedings of the 26th Symposium on Operating Systems Principles (SOSP'17)*. Association for Computing Machinery, New York, NY, 342–358. <https://doi.org/10.1145/3132747.3132771>
- [38] John Rushby. 1999. *Partitioning in Avionics Architectures: Requirements, Mechanisms, and Assurance*. Technical Report NASA/CR-1999-209347. <https://doi.org/10.5555/886690>
- [39] Peter Sewell, Susmit Sarkar, Scott Owens, Francesco Zappa Nardelli, and Magnus O. Myreen. 2010. X86-TSO: A rigorous and usable programmer's model for X86 multiprocessors. *Commun. ACM* 53, 7 (July 2010), 89–97. <https://doi.org/10.1145/1785414.1785443>
- [40] Philippe Stellwag, Jakob Krainz, and Wolfgang Schröder-Preikschat. 2010. A waitfree dynamic storage allocator by adopting the helping queue pattern. In *Proceedings of the 9th IASTED International Conference on Parallel and Distributed Computing and Networks (Innsbruck, Austria)*. ACTA Press, Calgary, AB, Canada, 79–87. <https://doi.org/10.2316/P.2010.676-052>
- [41] Shahar Timnat and Erez Petrank. 2014. A practical wait-free simulation for lock-free data structures. In *Proceedings of the 19th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP'14)*. Association for Computing Machinery, New York, NY, 357–368. <https://doi.org/10.1145/2555243.2555261>
- [42] Haosen Wen, Joseph Izraelevitz, Wentao Cai, H. Alan Beadle, and Michael L. Scott. 2018. Interval-based memory reclamation. In *Proceedings of the 23rd ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP'18)*. Association for Computing Machinery, New York, NY, 1–13. <https://doi.org/10.1145/3178487.3178488>
- [43] Steven Cameron Woo, Moriyoshi Ohara, Evan Torrie, Jaswinder Pal Singh, and Anoop Gupta. 1995. The SPLASH-2 programs: Characterization and methodological considerations. In *Proceedings of the 22nd Annual International Symposium on Computer Architecture (ISCA'95)*. Association for Computing Machinery, New York, NY, 24–36. <https://doi.org/10.1145/223982.223990>
- [44] Chaoran Yang and John Mellor-Crummey. 2016. A wait-free queue as fast as fetch-and-add. In *Proceedings of the 21st ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP'16)*. Association for Computing Machinery, New York, NY, Article 16, 13 pages. <https://doi.org/10.1145/2851141.2851168>
- [45] H. Yun, R. Mancuso, Z. Wu, and R. Pellizzoni. 2014. PALLOC: DRAM bank-aware memory allocator for performance isolation on multicore platforms. In *Proceedings of the IEEE 19th Real-Time and Embedded Technology and Applications Symposium (RTAS'14)*. 155–166. <https://doi.org/10.1109/RTAS.2014.6925999>
- [46] Yongwang Zhao and David Sanán. 2019. Rely-guarantee reasoning about concurrent memory management in zephyr RTOS. In *Proceedings of the 31st International Conference on Computer Aided Verification (CAV'19)*, New York City, NY, USA, July 15–18, 2019, *Proceedings, Part II (Lecture Notes in Computer Science)*, Isil Dillig and Serdar Tasiran (Eds.), Vol. 11562. Springer, 515–533. https://doi.org/10.1007/978-3-030-25543-5_29

Received July 2021; revised April 2022; accepted April 2022